

L^AT_EX.js Showcase

made with ♥ by Michael Brade

2017–2021

Abstract

This document will show most of the features of L^AT_EX.js while at the same time being a gentle introduction to L^AT_EX. In the appendix, the API as well as the format of custom macro definitions in JavaScript will be explained.

1 Characters

It is possible to input any UTF-8 character either directly or by character code using one of the following:

- `\symbol{"00A9}`: ©
- `\char"A9`: ©
- `^^A9` or `~~~~00A9`: © or ©

Special characters, like those:

`$ & % # _ { } ~ ^ \`

have to be escaped. More than 200 symbols are accessible through macros. For instance: 30°C is 86°F. See also Section 11.

2 Spaces and Comments

Spaces and comments, of course, work just as they do in L^AT_EX. This is an example: Supercalifragilisticexpialidocious It does not matter whether you enter one or several spaces after a word, it will always be typeset as one space—unless you force several spaces, like now. New T_EXusers may miss whitespaces after a command. Experienced T_EX users are T_EXperts, and know how to use whitespaces. Longer comments can be embedded in the `comment` environment: This is another example for embedding comments in your document.

3 Dashes and Hyphens

L^AT_EX knows four kinds of dashes. Access three of them with different numbers of consecutive dashes. The fourth sign is actually not a dash at all—it is the mathematical minus sign:

daughter-in-law, X-rated
pages 13–67
yes—or no?
0, 1 and –1

The names for these dashes are: ‘-’ hyphen, ‘—’ en-dash, ‘—’ em-dash, and ‘-’ minus sign. $\LaTeX$.js outputs the actual true unicode character for those instead of using the hyphen-minus.

4 Text and Paragraphs, Ligatures

An empty line starts a new paragraph, and so does `\par`.

Like this. A new line usually starts automatically when the previous one is full. However, using `\newline` or `\\`, one can force

to start a new line. Ligatures are supported as well, for instance:

fi, fl, ff, ffi, ffl ... instead of fi, fl, ffl ...

Use `\/` to prevent a ligature.

4.1 Multicolumns

The multi-column layout, using the `multicols` environment, allows easy definition of multiple columns of text—just like in newspapers. The first and mandatory argument specifies the number of columns the text should be divided into. It is often convenient to spread some text over all columns, just before the multicolumn

output. In $\LaTeX$, this was needed to prevent any page break in between. To achieve this, the `multicols` environment has an optional second argument which can be used for this purpose. For instance, this text you are reading now was started with the argument `\subsection{Multicolumns}`.

5 Boxes

$\LaTeX$.js supports most of the standard $\LaTeX$ boxes.

`\mbox{text}`

We already know one of them: it’s called `\mbox`. It simply packs up a series of boxes into another one, and can be used to prevent $\LaTeX$.js from breaking two words. As boxes can be put inside boxes, these horizontal box packers give you ultimate flexibility.

`\makebox[width][pos]{text}`

`\framebox[width][pos]{text}`

width defines the width of the resulting box as seen from the outside. The *pos* parameter takes a one letter value: center, flushleft, or flushright. `spread` is not really working in HTML. The command `framebox` works exactly the same as `makebox`, but it draws a box around the text. The following example shows you some things you could do with the `makebox` and `framebox` commands.

c e n t r a l

Guess I'm framed now!

Bummer, I am too wide

never find you and this?

`\parbox[pos] [height] [inner-pos] {width}{text}`

The `\parbox` command produces a box the contents of which are created in paragraph mode. However, only small pieces of text should be used, paragraph-making environments shouldn't be used inside a `\parbox` argument. For larger pieces of text, including ones containing a paragraph-making environment, you should use a `minipage` environment. By default LaTeX will position vertically a parbox so its center lines up with the center of the surrounding text line. When the optional position argument is present and equal either to 't' or 'b', this allows you respectively to align either the top or bottom line in the parbox with the baseline of the surrounding text. You may also specify 'm' for position to get the default behaviour. The optional *height* argument overrides the natural height of the box. The *inner-pos* argument controls the placement of the text inside the box, as follows; if it is not specified, *pos* is used. `t` text is placed at the top of the box. `c` text is centered in the box. `b` text is placed at the bottom of the box. `s` is not supported in HTML

The following examples demonstrate simple positioning:

- simple alignment: Some text

parbox
default
alignment,
parbox test
text

some text

parbox top
alignment,
text parbox
test text

some text

parbox
bottom
alignment,
text parbox
test text

some text.

alignment with a given height: Some text

parbox
default
alignment,
parbox test
text

some text

parbox top
alignment,
text parbox
test text

some text

parbox
bottom
alignment,
text parbox
test text

 some text.

The following examples demonstrate all explicit *pos/inner-pos* combinations:

- center alignment: Some text

parbox
default
alignment,
parbox test
text

 some text

parbox top
alignment,
text parbox
test text

some text

parbox
bottom
alignment,
text parbox
test text

 some text.

- top alignment: Some text

parbox
default
alignment,
parbox test
text

 some text

parbox top
alignment,
text parbox
test text

 some

text

parbox
bottom
alignment,
text parbox
test text

 some text.

- bottom alignment: Some text

parbox
default
alignment,
parbox test
text

 some text

parbox top
alignment,
text parbox
test text

some text parbox
bottom
alignment,
text parbox
test text some text.

- top/bottom in one line: Some text parbox
default
alignment,
parbox test
text some text parbox top
alignment,
text parbox
test text

some text.

5.1 Low-level box-interface

L^AT_EX.js supports the following low-level T_EX commands: `\llap{text}`, `\rlap{text}`, and `\smash{text}`, as well as `\hphantom{text}`, `\vphantom{text}`, and `\phantom{text}`.

A phantom looks like this: yes, now the phantom is ~~visible~~ `\llap` could be used to put something in the margin. However, there are better alternatives for that. See `\marginpar`.

test in
margin here

6 Spacing

The following horizontal spaces are supported:

Negative thin space: ||
 No space (natural): ||
 Thin space: || or ||
 Normal space: | |
 Normal space: | |
 Non-break space: | |
 en-space: | |
 em-space: | |
 2x em-space: | |
 3cm horizontal space: | |

7 Environments

7.1 Lists: Itemize, Enumerate, and Description

The `itemize` environment is suitable for simple lists, the `enumerate` environment for enumerated lists, and the `description` environment for descriptions.

1. You can nest the list environments to your taste:

- But it might start to look silly.
 - With a dash.

2. Therefore remember:

Stupid things will not become smart because they are in a list.

Smart things, though, can be presented beautifully in a list.

important Technical note: Viewing this in Chrome, however, will show too much vertical space at the end of a nested environment (see above). On top of that, margin collapsing for inline-block boxes is not allowed. Maybe using `dl` elements is too complicated for this and a simple nested `div` should be used instead.

Lists can be deeply nested:

- list text, level one
 - list text, level two
 - * list text, level three And a new paragraph can be started, too.
 - list text, level four And a new paragraph can be started, too.
This is the maximum level.
 - list text, level four
 - * list text, level three
 - list text, level two
- list text, level one
- list text, level one

7.2 Flushleft, Flushright, and Center

The `flushleft` environment:

This text is
left-aligned. $\text{\LaTeX}$ is not trying to make each line the same length.

The `flushright` environment:

This text is right-
aligned. $\text{\LaTeX}$ is not trying to make each line the same length.

And the `center` environment:

At the centre
of the earth

7.3 Quote, Quotation, and Verse

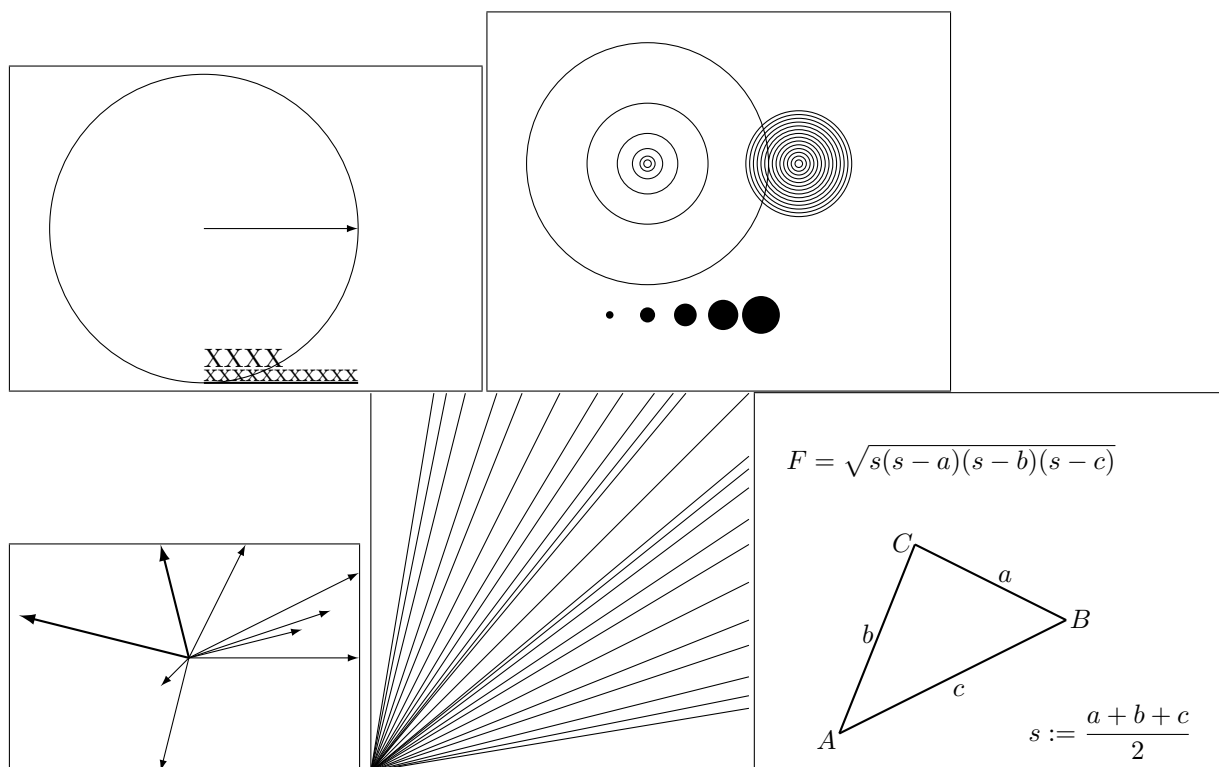
The `quote` environment is useful for quotes, important phrases and examples. A typographical rule of thumb for the line length is:

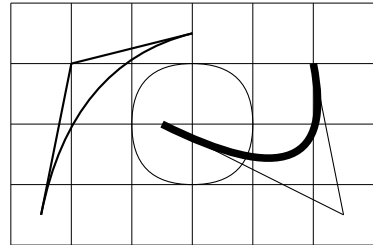
On average, no line should be longer than 66 characters.

There are two similar environments: the `quotation` and the `verse` environments. The `quotation` environment is useful for longer quotes going over several paragraphs, because it indents the first line of each paragraph. The `verse` environment is useful for poems where the line breaks are important. The lines are separated by issuing a `\\` at the end of a line and an empty line after each verse.

Humpty Dumpty sat on a wall:
 Humpty Dumpty had a great fall.
 All the King's horses and all the King's men
 Couldn't put Humpty together again. —J.W. Elliott

7.4 Picture





12 Fonts

Usually, L^AT_EX.js chooses the right font—just like L^AT_EX. In some cases, one might like to change fonts and sizes by hand. To do this, use the standard commands. The actual size of each font is a design issue and depends on the document class (in this case on the CSS file). The small and **bold** Romans ruled all of great big *Italy*. You can also emphasize text if it is set in italics, in a *sans-serif* font, or in *typewriter* style. The environment form of the font commands is available, too:

This whole paragraph is emphasized, for instance.

12.1 An advice

Remember! The M**O**R**E** fonts **YOU** use in a document, *the* more READABLE and *beautiful it becomes*.

A Source

The source of L^AT_EX.js is here on GitHub: <https://github.com/michael-brade/LaTeX.js>